Experiment No.07

Aim: Build an Artificial Neural Network (ANN) using Keras/TensorFlow.

Theory:

1. Dataset Source

Dataset: Churn Modelling Dataset

Source link:
https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling

2. Dataset Description

The dataset contains customer information from a bank, used to predict whether a customer will leave (churn) or not.

Features:

Feature	Description
CreditScore	Customer's credit score
Geography	Country (France, Spain, Germany)
Gender	Male/Female
Age	Customer age
Tenure	Years with bank
Balance	Account balance
NumOfProducts	Number of products used
HasCrCard	Has credit card (0/1)
IsActiveMember	Active status (0/1)
EstimatedSalary	Estimated salary
Exited	Target variable (1 = churn, 0 = not churn)

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of layers of neurons performing weighted transformations.

Neuron computation:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$a = f(z)$$

Activation Functions

ReLU:

$$f(x) = \max(0, x)$$

Sigmoid (Output layer):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Loss Function (Binary Cross Entropy)

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

Optimization

Weights are updated using Adam optimizer, which applies gradient descent:

$$w = w - \eta \frac{\partial L}{\partial w}$$

4. Algorithm Limitations

- Requires large dataset for better generalization
- Sensitive to feature scaling
- Black-box model (low interpretability)
- Performance affected by class imbalance
- Risk of overfitting without regularization
- Computationally expensive compared to simple models

5. Methodology / Workflow

Steps followed:

1. Load dataset (Churn_Modelling.csv)
2. Remove irrelevant columns (ID, Name)
3. Convert categorical variables using encoding
4. Separate features and target variable
5. Split dataset into training and testing sets
6. Apply StandardScaler for normalization
7. Build ANN model using Keras:
    - Input layer
    - Hidden layers with ReLU
    - Dropout layers for regularization
    - Output layer with sigmoid
8. Compile model using:
    - Optimizer: Adam
    - Loss: Binary Crossentropy
9. Train model with epochs and batch size
10. Evaluate model using:
    - Accuracy
    - Confusion Matrix
    - Classification Report
    - ROC-AUC
11. Plot:

    - Training vs validation graphs
    - ROC curve
    - Confusion matrix

6. Performance Analysis

Metrics Used:

- Accuracy
- Confusion Matrix
- Precision, Recall, F1-score
- ROC-AUC Score

Observations:

- Model achieves ~80% accuracy
- High accuracy for non-churn customers
- Lower recall for churn class due to imbalance
- ROC-AUC shows moderate prediction capability

Interpretation:

- ANN successfully captures customer behavior patterns
- Important features include:
    - Credit score
    - Balance
    - Activity status
- Model tends to favor majority class (non-churn)

7. Hyperparameter Tuning

Parameters Tuned:

Parameter	Values Tested
Epochs	20, 25, 30
Batch Size	16, 32
Hidden Layers	2–4 layers
Neurons	8, 16, 32, 64

Dropout	0.2 – 0.4

Final Configuration:

- Hidden Layers: 3
- Neurons: 32 → 16 → 8
- Dropout: 0.3
- Epochs: 25
- Batch Size: 32

Impact of Tuning:

- Improved generalization
- Reduced overfitting
- Stabilized training accuracy
- Better validation performance

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report,
roc_auc_score, roc_curve

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

# Load Dataset
data = pd.read_csv("sample_data/Churn_Modelling.csv")
print("Dataset size:", data.shape)
# Data Preprocessing
# Remove unnecessary columns
data = data.drop(["RowNumber", "CustomerId", "Surname"], axis=1)
# Convert categorical to numeric
data = pd.get_dummies(data, columns=["Geography", "Gender"], drop_first=True)
```

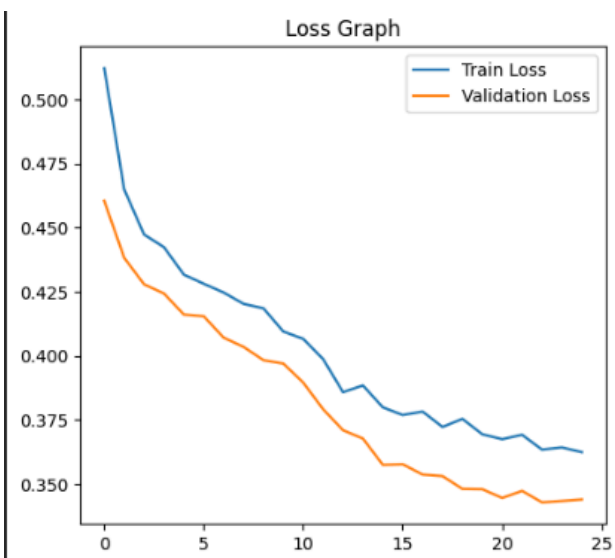
```
# Features & Target
X = data.drop("Exited", axis=1)
y = data["Exited"]
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=7,
    stratify=y
)
# Feature Scaling
sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)
# Build ANN Model
model = Sequential()

model.add(Dense(32, activation='relu', input_dim=X_train_scaled.shape[1]))
model.add(Dropout(0.3))

model.add(Dense(16, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(8, activation='relu'))

model.add(Dense(1, activation='sigmoid'))

# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Train Model
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=25,
    batch_size=32,
    verbose=1
```

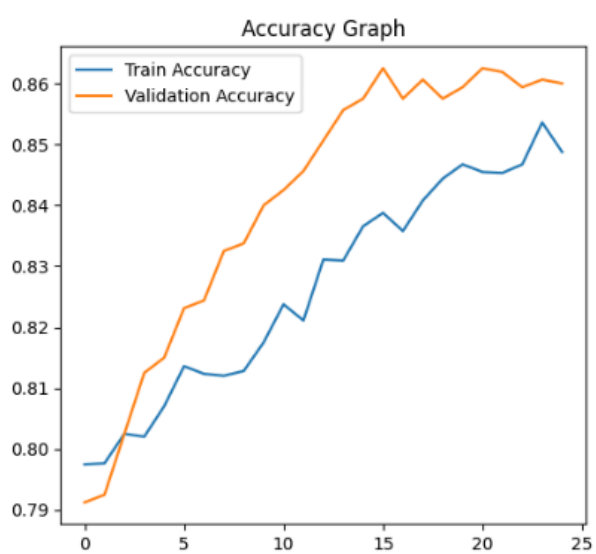
```
)

# Plot Training Graphs
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Graph")
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy Graph")
plt.legend()
plt.show()
# Predictions
y_prob = model.predict(X_test_scaled)
y_pred = (y_prob > 0.5).astype(int)
# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
# Confusion Matrix Plot
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap="coolwarm")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("ANN Confusion Matrix")
plt.show()
# ROC Curve
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label="ANN Model")
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
```

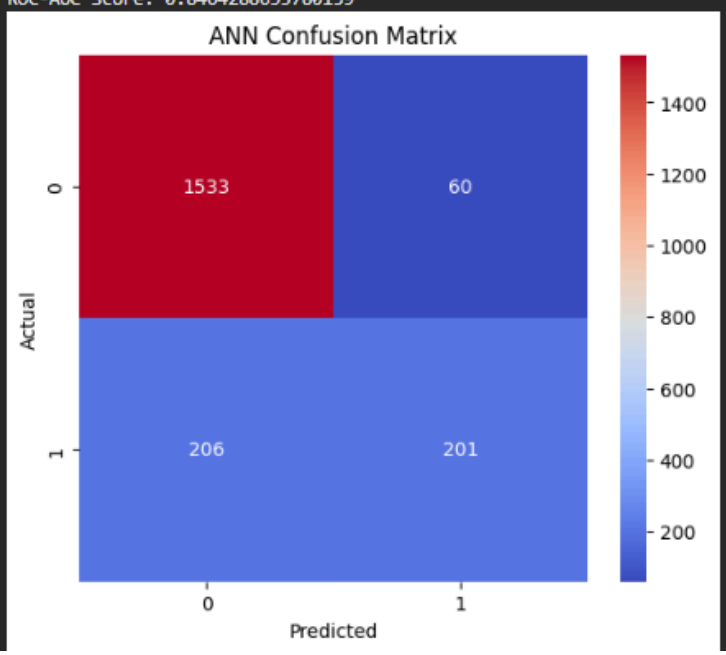
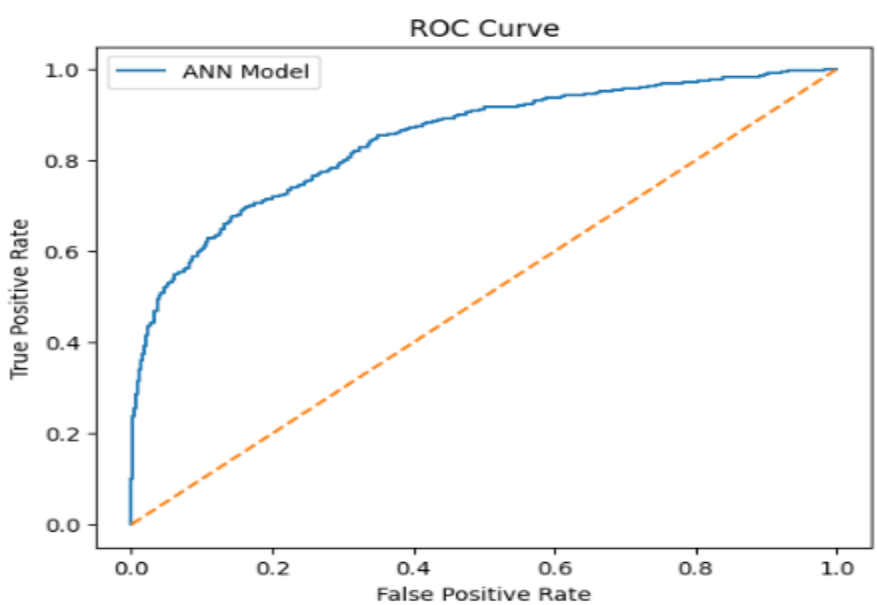
plt.show()

Output:

```
Dataset size: (10000, 14)
Epoch 1/25
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
200/200 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.7975 - loss: 0.5123 - val_accuracy: 0.7912 - val_loss: 0.4605
Epoch 2/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7977 - loss: 0.4650 - val_accuracy: 0.7925 - val_loss: 0.4383
Epoch 3/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8025 - loss: 0.4474 - val_accuracy: 0.8025 - val_loss: 0.4280
Epoch 4/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8020 - loss: 0.4425 - val_accuracy: 0.8125 - val_loss: 0.4244
Epoch 5/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8070 - loss: 0.4318 - val_accuracy: 0.8150 - val_loss: 0.4162
Epoch 6/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8136 - loss: 0.4282 - val_accuracy: 0.8231 - val_loss: 0.4155
Epoch 7/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8123 - loss: 0.4248 - val_accuracy: 0.8244 - val_loss: 0.4072
Epoch 8/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8120 - loss: 0.4204 - val_accuracy: 0.8325 - val_loss: 0.4036
Epoch 9/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8128 - loss: 0.4186 - val_accuracy: 0.8338 - val_loss: 0.3984
Epoch 10/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8175 - loss: 0.4096 - val_accuracy: 0.8400 - val_loss: 0.3970
Epoch 11/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8238 - loss: 0.4067 - val_accuracy: 0.8425 - val_loss: 0.3896
Epoch 12/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8211 - loss: 0.3988 - val_accuracy: 0.8456 - val_loss: 0.3792
Epoch 13/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8311 - loss: 0.3859 - val_accuracy: 0.8506 - val_loss: 0.3711
Epoch 14/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8309 - loss: 0.3885 - val_accuracy: 0.8556 - val_loss: 0.3678
Epoch 15/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8366 - loss: 0.3800 - val_accuracy: 0.8575 - val_loss: 0.3576
Epoch 16/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8388 - loss: 0.3770 - val_accuracy: 0.8625 - val_loss: 0.3578
Epoch 17/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8358 - loss: 0.3783 - val_accuracy: 0.8575 - val_loss: 0.3538
Epoch 18/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8408 - loss: 0.3723 - val_accuracy: 0.8606 - val_loss: 0.3532
Epoch 19/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8444 - loss: 0.3755 - val_accuracy: 0.8575 - val_loss: 0.3482
Epoch 20/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8467 - loss: 0.3695 - val_accuracy: 0.8594 - val_loss: 0.3481
Epoch 21/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8455 - loss: 0.3676 - val_accuracy: 0.8625 - val_loss: 0.3446
Epoch 22/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8453 - loss: 0.3693 - val_accuracy: 0.8619 - val_loss: 0.3474
Epoch 23/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8467 - loss: 0.3635 - val_accuracy: 0.8594 - val_loss: 0.3429
Epoch 24/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8536 - loss: 0.3643 - val_accuracy: 0.8606 - val_loss: 0.3434
Epoch 25/25
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8487 - loss: 0.3625 - val_accuracy: 0.8600 - val_loss: 0.3440
```






```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Accuracy: 0.867
```

```
Confusion Matrix:
 [[1533   60]
 [ 206  201]]

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.96      0.92      1593
           1       0.77      0.49      0.60       407

    accuracy                           0.87      2000
   macro avg       0.83      0.73      0.76      2000
weighted avg       0.86      0.87      0.86      2000

ROC-AUC Score: 0.8464288633780159
```